

UNIT III Software Design and Architecture

Syllabus

Design Process, Software design principles, Design Concepts, The Design Model, Software Architecture, architectural Genres, user interface design principles, Case Study: Library Management System.

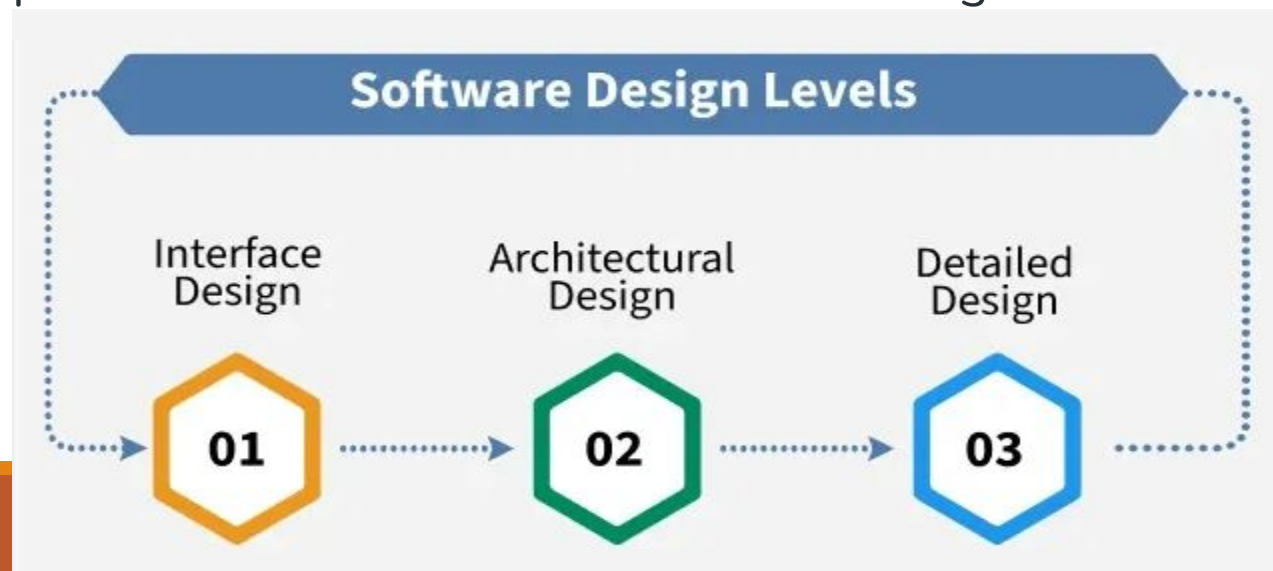
Software Design Process

- ❑ The **Design Phase** of software development deals with transforming the customer requirements as described in the SRS (Software Requirement Specification) documents into a form implementable using a programming language.

What is Software Design Process?

- ❑ **Software Design Process** is the phase where developers plan how to turn a set of requirements into a working system. Like a blueprint for the software. Instead of going straight into writing code, developers break down complex requirements into smaller, manageable pieces, design the system architecture, and decide how everything will fit together and work.

The software design process can be divided into the following three levels or phases of design:



Interface Design

Definition:

Interface design specifies **how components, modules, or systems communicate** with each other and with users.

Types of Interfaces

User interfaces (UI screens, dashboards)

Software interfaces (APIs, function calls)

Hardware interfaces (device communication)

External system interfaces (payment gateways, third-party services)

What It Defines

Input/output formats

Method signatures

Communication protocols

Error handling rules

Example:

Defining an API endpoint:

POST /login

Input: username, password

Output: authentication token

Architectural Design

Definition:

Architectural design defines the **overall system structure** and identifies major components and how they interact.

Key Objectives

Decide system architecture style.

Divide system into subsystems/modules

Define communication paths

Ensure scalability, performance, and reliability

Typical Artifacts

Architecture diagrams

Deployment diagrams

Technology selection

Module decomposition

Example:

In an online banking system:

Web client

Application server

Database server

and how they communicate.

3. Detailed Design

Definition:

Detailed design describes **internal logic of each module** so developers can directly implement it.

Key Elements

Algorithms

Data structures

Class design

Function logic

Pseudocode

Database queries

Goal:

Remove ambiguity so coding becomes straightforward.

Example:

Specifying the exact algorithm for password validation and encryption.

Software Design Phases

The Actual Software Design process travel through the one end to another end of making and Software well with completion of requirements and the phases which included in the same that are bellow:



1. Understanding Project Requirements

Before jumping into the design, the first step is to make sure the team understands what the users need, what the business goals are, and any potential challenges. This understanding sets the foundation for creating a design that meets both user and business needs.

2. Research, Analysis, and Planning

During this phase, the team gathers data through methods like interviews, surveys, and focus groups. This helps get a clearer picture of what the users want and allows the team to design with the user in mind, ensuring the software will truly meet their needs.

3. Designing the Software

In the design phase, the team starts creating visual elements like wireframes, user stories, and flow diagrams to map out how the system will work. Based on the feedback, prototypes are created and fine-tuned to make sure the design is moving in the right direction.

4. Technical Design

After gathering feedback, the team gets into more details with the technical design. This phase involves creating a thorough technical document that outlines exactly how the software will be implemented, including specific components and how they will work together.

5. User Interface Design

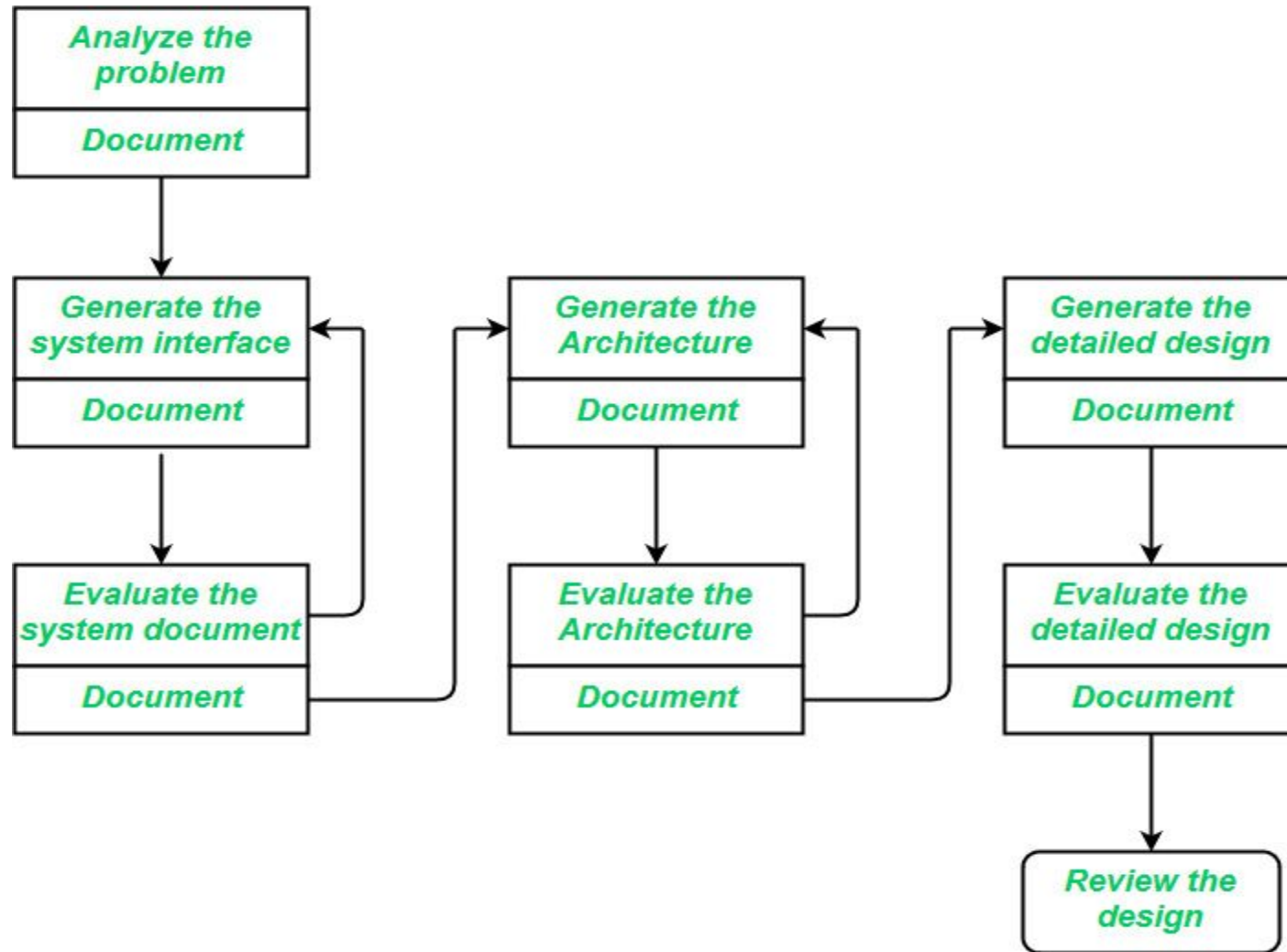
The focus here is on making sure the software is easy to use. UI designers work on the visual elements, navigation, and overall user experience to ensure the interface is intuitive and friendly.

6. Prototyping

Prototypes are created to visualize the design and functionality before the full development process begins. These can range from simple wireframes (low fidelity) to fully interactive models (high fidelity) depending on the stage of development.

Elements of Software Design

Good software design is built on several core element that work together to create effective system.



1. Architecture: This is the conceptual model that defines the structure, behavior, and views of a system. We can use flowcharts to represent and illustrate the architecture.

A solid architecture verify the system is flexible, stable, and easy to maintain over time.

2. Modules: Modules as the building blocks of the system. Each one handles a specific task or feature. Breaking a system into smaller modules makes it easier to develop, test, and maintain the system.

These are components that handle one specific task in a system. A combination of the modules makes up the system.

3. Components: This provides a particular function or group of related functions. They are made up of modules. Organising the system into components helps keep the code clean and makes the system more adaptable.

4. Interfaces: These are smaller units within modules that focus on specific functions. This is the shared boundary across which the components of a system exchange information and relate.

5. Data: Data is at the heart of any system. It's all about how information is stored, accessed, and shared. This is the management of the information and data flow.

How Software Design Fits into the SDLC?

After **planning**, the designing of software phase starts. This is where the team uses tools like wireframes, data flow diagrams, and UI designs to map out how the system will work. It's like drawing the roadmap for how everything will come together and how we can use each component well with proper functionality. After the Designing phase the Actual process of Software Development begins.

Software Design Principles

To make sure software is easy to manage, maintain, and update, there are a few important principles that should guide the design process:

- 1.Modularity:** Think of modularity as breaking down the software into smaller, independent sections or "**modules**". Each module handles a specific task, which makes it much easier to test, maintain, and update without affecting the entire system.
- 2.Coupling:** Coupling refers to how much one part of the software depends on others. The goal is to keep the connections between modules to a minimum, so that changing one module won't impact others. This makes the software more flexible and easier to update in the future.

- 3.Abstraction:** Abstraction is about simplifying the software by hiding its complexity. It only shows users the essential features they need, making the software easier to use and understand, without overwhelming them with unnecessary details.
- 4.Anticipation of Change:** It's important to design software with future changes in mind. This means building it in a way that allows for easy updates or adjustments, whether it's adding new features or adapting to new technologies.
- 5.Simplicity:** Simple designs are the best. Keeping things simple means less room for errors, and it's easier to maintain and improve over time. The goal is to avoid overcomplicating things, keeping the design clean and efficient.
- 6.Sufficiency & Completeness:** The design should make sure the software meets all the required functions without unnecessary extras. It's about striking a balance making sure everything needed is there, but avoiding unnecessary features that can slow things down or create confusion.

Tools for Software Design

There are many tools that make the process easier and more efficient, whether you're working on wireframes, prototypes, or documentation. Here are some popular tools:

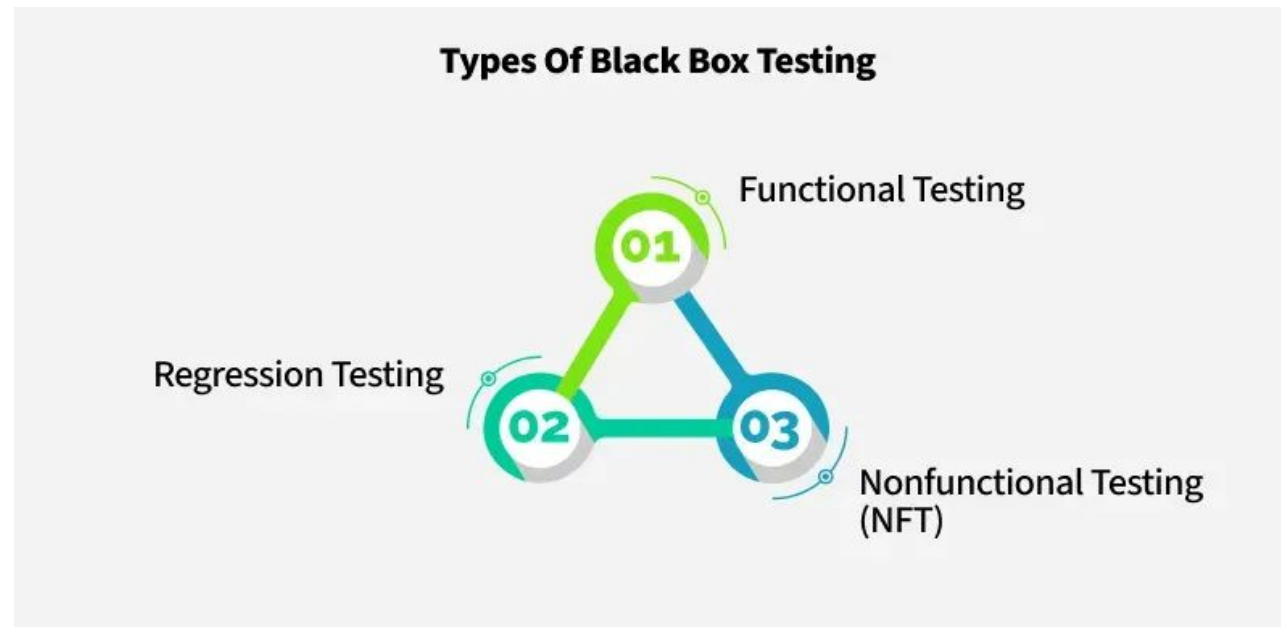
1. Figma
2. Balsamiq
3. Axure RP
4. Sketch
5. InVision Studio

Black Box Testing

Black-box testing is a Type of Software Testing in which the tester is not concerned with the software's internal knowledge or implementation details but rather focuses on validating the functionality based on the provided specifications or requirements.

Types Of Black Box Testing

The testing of application without knowing the internal code or structure, following are the various Types of Black Box Testing:



1. Functional Testing

- Functional Testing is a type of Software Testing in which the system is tested against the functional requirements and specifications.
- Functional testing ensures that the requirements or specifications are properly satisfied by the application.
- This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.
- This testing focuses on checking the user interface, APIs, database, security, client or server application, and functionality of the Application Under Test. Functional testing can be manual or automated. It determines the system's software functional requirements.

2. Regression Testing

- Regression Testing is like a Software Quality checkup after any changes are made. It involves running tests to make sure that everything still works as it should, even after updates or tweaks to the code. This ensures that the software remains reliable and functions properly, maintaining its integrity throughout its development lifecycle.
- Regression means the return of something and in the software field, it refers to the return of a bug. It ensures that the newly added code is compatible with the existing code.
- In other words, a new software update has no impact on the functionality of the software. This is carried out after a system maintenance operation and upgrades.

3. Nonfunctional Testing

- Non-functional Testing is a type of Software Testing that is performed to verify the non-functional requirements of the application. It verifies whether the behavior of the system is as per the requirement or not. It tests all the aspects that are not tested in functional testing.
- It focuses on the software's performance, usability, and scalability.

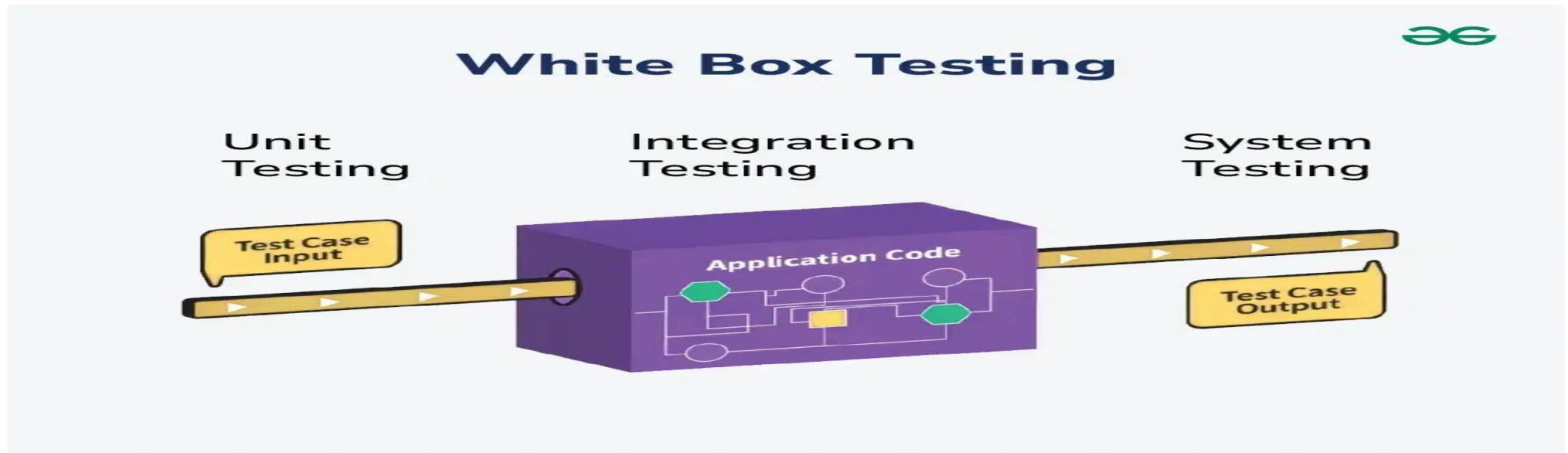
Advantages of Black Box Testing

- The tester does not need to have more functional knowledge or programming skills to implement the Black Box Testing.
- It is efficient for implementing the tests in the larger system.
- Tests are executed from the user's or client's point of view.
- Test cases are easily reproducible.
- It is used to find the ambiguity and contradictions in the functional specifications.

White box Testing

White box testing is a Software Testing Technique that involves testing the internal structure and workings of a Software Application.

- The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.
- White box testing is also known as **Structural Testing or Code-based Testing**, and it is used to test the software's internal logic, flow, and structure.
- The tester creates test cases to examine the code paths and logic flows to ensure they meet the specified requirements.



White-box Testing focuses on the internal workings of an application, ensuring that its logic, structure, and flow operate as intended. Unlike black-box testing, which focuses on user interactions without knowledge of the underlying code, white-box testing involves examining the software's source code directly.

1. Code Logic and Flow

Checks if the program's logic works as intended. This means verifying that code modules (like functions or classes) interact correctly and that control structures such as if-else statements, loops, or switches execute properly. For example, ensuring a login function redirects users correctly based on valid or invalid credentials.

2. Code Coverage

Ensures tests exercise as much of the code as possible. This includes:

- **Statement coverage:** Every line of code runs at least once.
- **Branch coverage:** All decision paths (e.g., true/false conditions) are tested.
- **Path coverage:** Every possible route through the code is checked.
This helps to find untested or “dead” code that could hide bugs.

3. Data Flow and Variables

Verifies that data is passed and manipulated correctly through the application. This includes ensuring variables are properly initialized, updated, and used without causing any errors or unexpected behavior. Monitoring the flow of data ensures that the system remains stable and reliable as it processes inputs, calculations, and outputs.

4. Internal Functions and Methods

Tests the individual functions or methods to ensure they perform their intended tasks accurately and return the expected results. This part of white-box testing focuses on validating business logic, mathematical computations, and other operations within the software. Ensuring these internal processes are correct helps catch potential issues early and improves the reliability of the overall system.

5. Boundary Conditions

Examines how the code handles edge cases, like the maximum or minimum values for inputs (e.g., a loop running 0 or 100 times, or an input field accepting a 255-character string). This ensures the app doesn't crash at its limits.

6. Error Handling and Exception Management

Confirms the program manages errors smoothly, catching exceptions (e.g., invalid inputs) and providing clear feedback. For example, testing if a file upload function handles a missing file gracefully.

Advantages of White Box Testing

Here are the Advantages of White Box Testing:

- **Thorough Testing:** White box testing is thorough as the entire code and structures are tested.
- **Code Optimization:** It results in the optimization of code removing errors and helps in removing extra lines of code.
- **Early Detection of Defects:** It can start at an earlier stage as it doesn't require any interface as in the case of black box testing.
- **Integration with SDLC:** White box testing can be easily started in Software Development Life Cycle.
- **Detection of Complex Defects:** Testers can identify defects that cannot be detected through other testing techniques.
- **Comprehensive Test Cases:** Testers can create more comprehensive and effective test cases that cover all code paths.

Software design principles

Software Design Principles are systematic guidelines used to design reliable, maintainable, and scalable software systems. They help developers structure code and architecture so that systems remain easy to understand, modify, test, and extend over time.

Software Design Principles

1. Modularity

Divide software into smaller independent modules where each module performs a specific task.

Benefit: Easier maintenance, testing, and reuse.

2. Abstraction

Show only essential features and hide internal implementation details.

Example: A user interacts with a login system without knowing how authentication works internally.

3. Encapsulation

Combine data and methods into a single unit (class) and restrict direct access to internal data.

Benefit: Protects data integrity and prevents unintended modifications.

4. Separation of Concerns (SoC)

Split system functionality into distinct sections where each section handles one concern.

Example layers:

- Presentation layer - Handles user interaction.
- Business logic layer - Contains core system rules and decision-making logic.
- Data access layer - Handles communication with the database.

1 Presentation Layer (UI Layer)

Responsibility:

Handles user interaction.

What it does:

- Displays data to users
- Accepts user input
- Sends user requests to business layer
- Does NOT contain business rules

Examples:

- Web pages (HTML, CSS, JavaScript)
- Mobile app screens
- REST API controllers

Technologies:

- React, Angular
- JSP, ASP.NET
- Flask/Django views
- Spring MVC controllers

Example:

A login form that collects username and password.

② Business Logic Layer (BLL)

Responsibility:

Contains core system rules and decision-making logic.

What it does:

- Validates input
- Applies business rules
- Performs calculations
- Processes transactions
- Controls application flow

Example:

- Check if password is correct
- Calculate tax
- Check if user has enough balance
- Fraud detection logic

This is the **heart of the system**.

3 Data Access Layer (DAL)

Responsibility:

Handles communication with the database.

What it does:

- Executes SQL queries
- Inserts, updates, deletes records
- Retrieves data
- Maps database records to objects

Example:

- `SELECT * FROM users WHERE id = 10`
- `INSERT INTO orders VALUES (...)`

Technologies:

- JDBC
- Hibernate
- JPA
- Entity Framework
- SQLAlchemy

5. Low Coupling

Modules should have minimal dependency on each other.

Why important: Changes in one module should not heavily affect others.

6. High Cohesion

Each module should focus on a single, well-defined responsibility.

Result: More understandable and reusable components.

7. DRY (Don't Repeat Yourself)

Avoid duplication of code or logic.

Approach: Use functions, classes, or libraries for repeated operations.

8. KISS (Keep It Simple, Stupid)

Prefer simple solutions over complex ones.

Reason: Simple designs are easier to debug, maintain, and scale.

9. YAGNI (You Aren't Gonna Need It)

Do not add features unless they are actually required.

Goal: Avoid unnecessary complexity and over-engineering.

10. SOLID Principles (Object-Oriented Design Core)

Principle	Description
Single Responsibility	One class should do one thing
Open/Closed	Extend behavior without modifying existing code
Liskov Substitution	Subclasses must behave like parent classes
Interface Segregation	Prefer many small interfaces
Dependency Inversion	Depend on abstractions, not implementations

SOLID Principles

SOLID is a set of five object-oriented design principles introduced by **Robert C. Martin** to make software systems more understandable, flexible, and maintainable. Each principle targets a specific design problem.

S — Single Responsibility Principle (SRP)

Definition

A class should have **only one reason to change**, meaning it should perform only one job or responsibility.

Why it matters

If a class handles multiple tasks, changes in one responsibility may break others.

Bad Example

A `Report` class that:

- Generates report data
- Formats report
- Saves to file

This class mixes **business logic + formatting + storage**.

Good Example

Split into:

- ReportGenerator
- ReportFormatter
- ReportSaver

Each class has one responsibility.

Benefits

- Easier debugging
- Better readability
- Easier testing

O — Open/Closed Principle (OCP)

Definition

Software entities should be:

Open for extension but closed for modification

You should be able to add new behavior without changing existing code.

Problem it solves

Changing old code increases risk of introducing bugs.

Example

Instead of modifying a payment class every time you add a payment method:

 Bad

```
if payment_type == "card"  
elif payment_type == "upi"  
elif payment_type == "wallet"
```

 Good

Create a base Payment interface and subclasses:

- CardPayment
- UPIPayment
- WalletPayment

New methods = new classes (no change to old code).

L — Liskov Substitution Principle (LSP)

Definition

Objects of a subclass should be replaceable with objects of the superclass **without breaking the program**.

Meaning

Derived classes must behave correctly when used as their base type.

Violation Example

Base class: `Bird` → `fly()`

Subclass: `Penguin`

Penguins can't fly → breaks expectation.

Fix

Separate hierarchy:

- `Bird`
- `FlyingBird`
- `SwimmingBird`

Now substitution works correctly.

I — Interface Segregation Principle (ISP)

Definition

Clients should not be forced to depend on methods they do not use.

Problem

Large interfaces force classes to implement unnecessary methods.

Bad Example

```
interface Machine:  
    print()  
    scan()  
    fax()
```

A simple printer must implement `scan()` and `fax()` even if unused.

Good Design

Split interfaces:

- Printer
- Scanner
- Fax

Classes implement only what they need.

D — Dependency Inversion Principle (DIP)

Definition

High-level modules should not depend on low-level modules. Both should depend on **abstractions**.

Idea

Depend on interfaces, not concrete classes.

Bad Design

```
OrderService → MySQLDatabase
```

If database changes → modify service.

What this means:

OrderService (business logic) is directly using MySQLDatabase. It is tightly connected to MySQL.

Good Design

```
OrderService → DatabaseInterface
```

```
DatabaseInterface ← MySQLDatabase
```

```
DatabaseInterface ← MongoDBDatabase
```

Now database can change without affecting business logic.

OrderService does NOT know about MySQL or MongoDB. It only knows about a Database interface. Any database that follows the interface can be used.

Real-World Analogy (Simple)

Think of a restaurant kitchen:

- SRP → Chef cooks, cashier bills, waiter serves
- OCP → Add new dishes without rebuilding kitchen
- LSP → Any chef can replace another chef
- ISP → Waiter doesn't need cooking skills
- DIP → Restaurant depends on ingredient suppliers, not specific farmers

SOLID principles help developers build software that is easy to extend, test, maintain, and scale without breaking existing functionality.

Software Design Process/Concept

Software Design is the process of transforming user requirements into a suitable form, which helps the programmer in software coding and implementation. During the software design phase, the design document is produced, based on the customer requirements as documented in the SRS document. Hence, this phase aims to transform the SRS document into a design document.

The following items are designed and documented during the design phase:

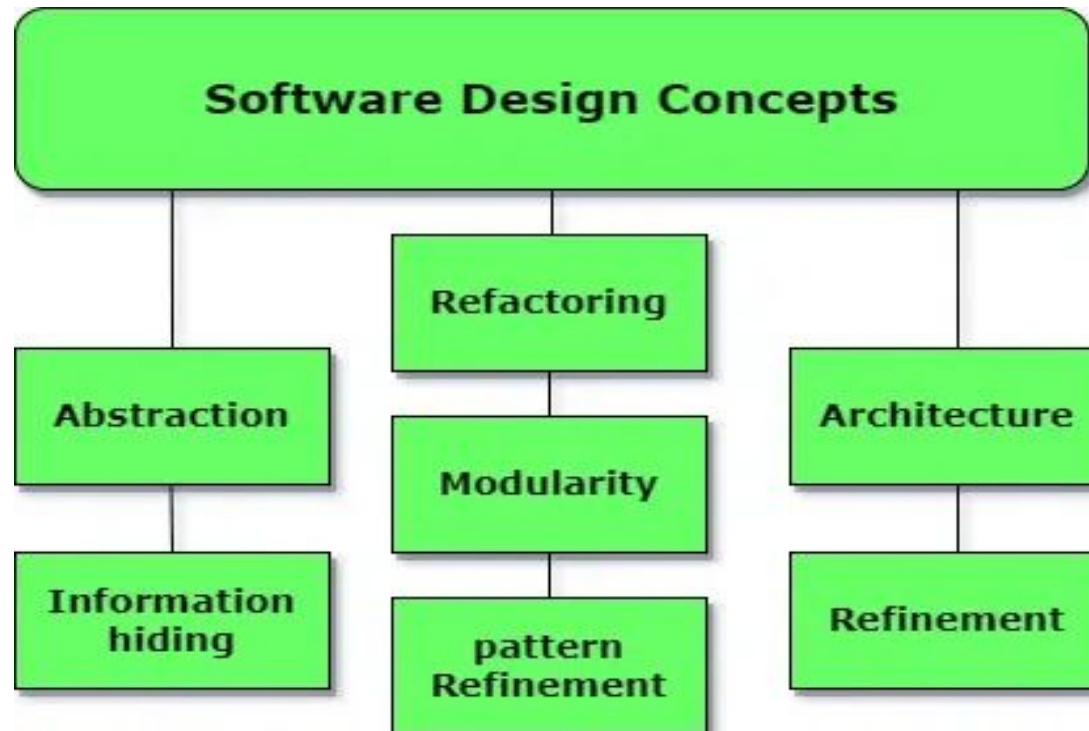
1. Different modules are required.
2. Control relationships among modules.
3. Interface among different modules.
4. Data structure among the different modules.
5. Algorithms are required to be implemented among the individual modules.

Objectives of Software Design

- 1. Correctness:** A good design should be correct i.e. , it should correctly implement all the functionalities of the system.
- 2. Efficiency:** A good software design should address the resources, time, and cost optimization issues.
- 3. Flexibility:** A good software design should have the ability to adapt and accommodate changes easily. It includes designing the software in a way, that allows for modifications, enhancements, and scalability without requiring significant rework or causing major disruptions to the existing functionality.
- 4. Understandability:** A good design should be easily understandable, it should be modular, and all the modules are arranged in layers.
- 5. Completeness:** The design should have all the components like data structures, modules, external interfaces, etc.
- 6. Maintainability:** A good software design aims to create a system that is easy to understand, modify, and maintain over time. This involves using modular and well-structured design principles e.g.,(employing appropriate naming conventions and providing clear documentation). Maintainability in Software and design also enables developers to fix bugs, enhance features, and adapt the software to changing requirements without excessive effort or introducing new issues.

Software Design Concepts

The **software design concept** simply means the idea or principle behind the design. It describes how you plan to solve the problem of designing software, and the logic, or thinking behind how you will design software. It allows the software engineer to create the model of the system software or product that is to be developed or built. The software design concept provides a supporting and essential structure or model for developing the right software. There are many concepts of software design and some of them are given below:



Points to be Considered While Designing Software

Abstraction (Hide Irrelevant data):

1. Abstraction simply means to hide the details to reduce complexity and increase efficiency or quality.
2. Different levels of Abstraction are necessary and must be applied at each stage of the design process so that any error that is present can be removed to increase the efficiency of the software solution and to refine the software solution.

Modularity (subdivide the system):

1. Modularity simply means dividing the system or project into smaller parts to reduce the complexity of the system or project.
2. In the same way, modularity in design means subdividing a system into smaller parts so that these parts can be created independently and then use these parts in different systems to perform different functions.

- 3.Architecture (design a structure of something):** Architecture simply means a technique to design a structure of something. Architecture in designing software is a concept that focuses on various elements and the data of the structure. These components interact with each other and use the data of the structure in architecture.
- 4.Refinement (removes impurities):** Refinement simply means to refine something to remove any impurities if present and increase the quality. The refinement concept of software design is a process of developing or presenting the software or system in a detailed manner which means elaborating a system or software. Refinement is very necessary to find out any error if present and then to reduce it.
- 5.Pattern (a Repeated form):** A pattern simply means a repeated form or design in which the same shape is repeated several times to form a pattern. The pattern in the design process means the repetition of a solution to a common recurring problem within a certain context.
- 6.Refactoring (Reconstruct something):** Refactoring simply means reconstructing something in such a way that it does not affect the behavior of any other features. Refactoring in software design means reconstructing the design to reduce complexity and simplify it without impacting the behavior or its functions. Fowler has defined refactoring as “the process of changing a software system in a way that it won't impact the behavior of the design and improves the internal structure”.

Different levels of Software Design

There are three different levels of software design. They are:

- 1.Architectural Design:** The architecture of a system can be viewed as the overall structure of the system and the way in which structure provides conceptual integrity of the system. The architectural design identifies the software as a system with many components interacting with each other. At this level, the designers get the idea of the proposed solution domain.
- 2.Preliminary or high-level design:** Here the problem is decomposed into a set of modules, the control relationship among various modules identified, and also the interfaces among various modules are identified. The outcome of this stage is called the program architecture. Design representation techniques used in this stage are structure chart and UML.
- 3.Detailed design:** Once the high-level design is complete, a detailed design is undertaken. In detailed design, each module is examined carefully to design the data structure and algorithms. The stage outcome is documented in the form of a module specification document.

User Interface Design Principles in Software Engineering

User Interface (UI) design principles are guidelines used to create software interfaces that allow users to interact with systems effectively. A good interface should minimize user effort, reduce errors, and improve user satisfaction.

1. Consistency

Consistency means maintaining the **same design patterns, layout, terminology, and behavior** throughout the application . When users interact with software, they learn how the system behaves. If the interface changes frequently, users become confused. Consistency ensures that users do not need to relearn how to perform tasks in different parts of the system.

Types of Consistency

1. **Visual Consistency** – Same colors, fonts, icons, and layouts.
2. **Functional Consistency** – Same actions produce the same results.
3. **Internal Consistency** – All screens within the system follow similar patterns.

Example

If the **Submit button** is located at the bottom-right of one form, it should remain in the same position on other forms.

Benefits

- Reduces learning time
- Improves usability
- Increases user confidence

2. Simplicity

Simplicity means designing an interface that is **clear, uncluttered, and easy to understand**.

The system should present only the necessary information and controls required for the user to complete a task. Overly complex interfaces overwhelm users and increase the likelihood of mistakes.

Methods to Achieve Simplicity

- Use clear labels and instructions
- Avoid unnecessary graphics or text
- Group related information together
- Use minimal design elements

Example

A simple login page should contain only:

- Username field
- Password field
- Login button

Benefits

- Faster task completion
- Reduced cognitive load
- Better user experience

3. Visibility

Visibility refers to how easily users can **see and understand available options and system status**.

Users should always know:

- What actions they can perform
- What the system is currently doing
- What will happen next

Example

When downloading a file, a **progress bar** shows the current download status.

Good Visibility Includes

- Clear buttons and menus
- Status indicators
- Progress bars
- Notifications

Benefits

- Reduces confusion
- Improves interaction efficiency
- Keeps users informed

4. Feedback

Feedback means the system should **immediately respond to user actions** so users know their action has been recognized.

Without feedback, users may think the system is not working.

Types of Feedback

1. Visual Feedback – Color change, progress bars

2. Audio Feedback – Notification sounds

3. Text Feedback – Messages confirming actions

Examples

- “File uploaded successfully”
- Button changes color when clicked
- Loading animation appears when processing

Benefits

- Builds user confidence
- Prevents repeated actions
- Improves system transparency

5. Error Prevention and Handling

Good UI design should **prevent errors before they occur** and help users recover easily if errors happen.

Error Prevention Techniques

- Input validation
- Drop-down menus instead of manual typing
- Confirmation dialogs before critical actions

Example

If a user enters letters in a numeric field, the system should show:

“Please enter a valid number.”

Error Recovery Methods

- Provide clear error messages
- Suggest solutions
- Allow undo operations

Benefits

- Reduces frustration
- Improves system reliability
- Prevents data loss

6. Flexibility

Flexibility allows the interface to support **different types of users and interaction styles**. Some users prefer simple navigation while others prefer faster methods.

Methods to Provide Flexibility

- Keyboard shortcuts
- Customizable settings
- Multiple navigation methods

Example

A text editor allows users to:

- Click menu options
- Use keyboard shortcuts (Ctrl + C, Ctrl + V)

Benefits

- Supports beginners and expert users
- Improves efficiency
- Enhances usability

7. User Control

Users should always feel that **they control the system**, not the other way around. The interface should allow users to easily start, stop, or reverse actions.

Key Features

- Cancel options
- Undo/redo commands
- Back buttons
- Exit options

Example

In a photo editing software, users can press **Undo** to reverse the last change.

Benefits

- Prevents mistakes from becoming permanent
- Improves user confidence

8. Accessibility

Accessibility ensures that software can be used by **people with different physical abilities or limitations**.

Design should consider users with visual, hearing, or motor impairments.

Accessibility Features

- Screen reader compatibility
- High-contrast color schemes
- Large fonts
- Keyboard navigation

Example

A website allowing visually impaired users to navigate using **screen readers**.

Benefits

- Inclusive design
- Wider user reach
- Compliance with accessibility standards

9. Clear Navigation

Navigation helps users move through the system easily and find the information they need.

Poor navigation can make even a powerful system difficult to use.

Navigation Tools

- Menu bars
- Navigation panels
- Breadcrumbs
- Search functions

Example

E-commerce websites provide navigation like:

Home → Electronics → Mobile Phones

Benefits

- Faster information access
- Reduced confusion
- Improved user experience

10. Aesthetic Design

Aesthetic design focuses on making the interface **visually appealing and well-organized**.
An attractive interface increases user engagement and satisfaction.

Key Elements

- Balanced layout
- Readable typography
- Proper spacing
- Consistent color schemes

Example

Modern applications use **clean layouts, simple icons, and soft colors**.

Benefits

- Enhances usability
- Improves user satisfaction
- Creates positive first impressions

Shneiderman's 8 Golden Rules of User Interface Design

Shneiderman's 8 Golden Rules of User Interface Design were proposed by **Ben Shneiderman**, a well-known computer scientist in **Human–Computer Interaction**.

These rules provide guidelines for designing **effective, user-friendly interfaces** that improve usability and user satisfaction.

1. Strive for Consistency

Consistency means the interface should follow **uniform design patterns, terminology, colors, layouts, and actions** throughout the system.

Users learn how the system works through repeated interaction. If similar actions behave differently in different parts of the system, users become confused.

Example

- The **Save** button should appear in the same location on every form.
- The same icons and colors should represent the same functions across the application.

Benefits

- Reduces learning time
- Improves usability
- Increases user confidence

2. Enable Frequent Users to Use Shortcuts

Experienced users often want to perform tasks **quickly without navigating multiple menus**. Providing shortcuts allows expert users to increase their productivity.

Examples

- Keyboard shortcuts such as **Ctrl + C** (copy) and **Ctrl + V** (paste)
- Autocomplete features
- Function keys

Benefits

- Saves time for experienced users
- Improves efficiency
- Supports different skill levels

3. Offer Informative Feedback

The system should always **inform users about what is happening after an action.**

Every user action should produce some form of feedback so the user knows the system has responded.

Types of Feedback

- Visual (progress bars, loading icons)
- Text messages
- Audio alerts

Example

After submitting an online form, the system displays:

“Your form has been submitted successfully.”

Benefits

- Prevents confusion
- Builds user trust in the system

4. Design Dialogs to Yield Closure

Interactions should have a **clear beginning, middle, and end**.

After completing a task, the system should provide confirmation so users feel the process is complete.

Example

In an online shopping system:

1. Add product to cart
2. Enter payment details
3. Receive **“Order Placed Successfully”** message

This confirmation provides closure to the interaction.

Benefits

- Gives users a sense of completion
- Reduces uncertainty

5. Prevent Errors

The interface should be designed to **reduce the chances of users making mistakes**.

Instead of correcting errors later, the system should try to **prevent them from occurring**.

Techniques

- Use dropdown menus instead of manual typing
- Validate input fields
- Disable invalid options

Example

A form that prevents users from submitting if **mandatory fields are empty**.

Benefits

- Improves system reliability
- Reduces user frustration

6. Permit Easy Reversal of Actions

Users should be able to **undo or reverse their actions easily**.

This encourages users to explore the system without fear of making irreversible mistakes.

Examples

- **Undo / Redo buttons**
- Cancel options
- Delete confirmation dialogs

Example

In a text editor, pressing **Undo** restores previously deleted text.

Benefits

- Reduces user anxiety
- Encourages experimentation

7. Support Internal Locus of Control

Users should feel that **they control the system**, not the system controlling them. The interface should respond to user actions rather than forcing users into unexpected processes.

Examples

- Allow users to cancel operations
- Avoid unexpected pop-ups
- Provide navigation options

Benefits

- Improves user satisfaction
- Increases sense of control

8. Reduce Short-Term Memory Load

Human short-term memory is limited. Therefore, interfaces should **minimize the amount of information users must remember**.

Important information should always be visible rather than requiring users to recall it from memory.

Methods

- Use menus instead of memorizing commands
- Provide tooltips and instructions
- Display previously entered information

Example

A website displaying **recent searches** so users do not have to remember them.

Benefits

- Reduces cognitive load
- Improves usability